Robotics Project II

Pietro Bonazzi, Group 2

IN.2022 Robotics 2019, BSc Course, 6th Sem.
University of Fribourg
pietro.bonazzi@unifr.ch

Abstract

The following document presents two behaviors programmed on an epuck 2 and simulated in webots. A knowledge of Braitenberg's behaviors and of the functionalities of the robot itself is necessary to understand how the two controllers presented work. In both cases robots use different sensors to perform multiple activities. In the first case the robots will follow a line on the ground and will circumvent or stop in front of objects depending on their color. In the second situation, a robot will be scanning a labyrinth of physical walls. Using Dijkstra's algorithm the quickest solution to reach the exit of the maze from the initial point will be calculated. Then a second robot will use this solution to exit the maze as efficiently as possible.

Keywords: e-puck 2, webots, simulation, line follower, Dijkstra's algorithm, maze's solver

Contents

1 Introduction **2**

2 Task **3**
2.1 Problem statement . . . 3
2.2 Solution Strategy . . . 4
2.2.1 Line Follower . . . 4
2.2.2 Response in case of an obstacle . . . 5
2.2.3 Communication, loop and LEDs . . . 5
2.3 Implementation . . . 6
2.4 Validation . . . 7

3 Challenge **8**
3.1 Problem statement . . . 8
3.2 Solution Strategy . . . 8
3.2.1 Mapping . . . 9
3.2.2 Dijkstra's algorithm and relative path . . . 9
3.2.3 Communication and execution . . . 9
3.3 Implementation . . . 10
3.4 Validation . . . 11

4 Conclusion **12**

Appendix **14**
Appendix A Task . . . 14
A.1 Problem Statement . . . 14
A.2 LED's Recognition . . . 19
A.3 Images . . . 20
A.4 Functions . . . 21
A.5 Controller . . . 26
Appendix B Challenge . . . 28
B.1 Functions . . . 28
B.2 Controller . . . 34
Appendix C FSM Diagrams . . . 35
C.1 FSM - Task . . . 35
C.2 FSM - Challenge . . . 36

Chapter 1

Introduction

The goal of this project is to show two behaviors programmed on an e-puck 2 that use in sync different features offered by the robot itself. In chapters 2 and 3 it will be shown how position sensors, ground sensors, camera and communication channels can work in unison to perform complex behaviors. The first combination (chapter 2) of sensors will see several robots running in a track, dodging obstacles and coordinating with each other to perform LED choreographies. In the second combination (chapter 3) a robot will map a maze of physical walls using lines on the floor. Position sensors will be used to recognize barriers. After calculating the optimal path with Dijkstra's algorithm, the solution to the second robot will be sent. The latter will only have to follow the instructions. In the implementation of the two behaviors some difficulties have been encountered. In particular in the first controller the main challenge was to make the robot follow the lines on the ground in all tracks. On the other hand, the second robot's controller implemented to solve the maze may not explore special mazes with loops inside them. Overall, the results obtained are highly satisfactory and leave room for an external operator to make changes from the outside without affecting the reliability of the code.
In the first case the robot is actually able to follow adverse paths with narrow lines drawn by hand. In the second behavior, the external operator can modify the size of the labyrinth and the start and end coordinates at will.

Chapter 2

Task

This chapter is dedicated to the description of an e-puck 2's behavior. The structure of the problem was defined by Professor Julien Nembrini. The original statement is accessible to the reader in Appendix A of this document. After the description of the problem (2.1) , the resolution strategy will be discussed (2.2). Subsequently, the implementation will be presented (2.3). In the last section the results obtained will be shared (2.4).

2.1 Problem statement

According to the original statement, three e-puck robots are positioned in the middle of an arena and inside a circuit outlined by a black line on the ground. Above this line are placed three obstacles of red, green and blue color respectively (for a more detailed description of the RGB scale colors see the section 2.3). The figure below, taken from Appendix A, portrays the environment at the beginning of the simulation.

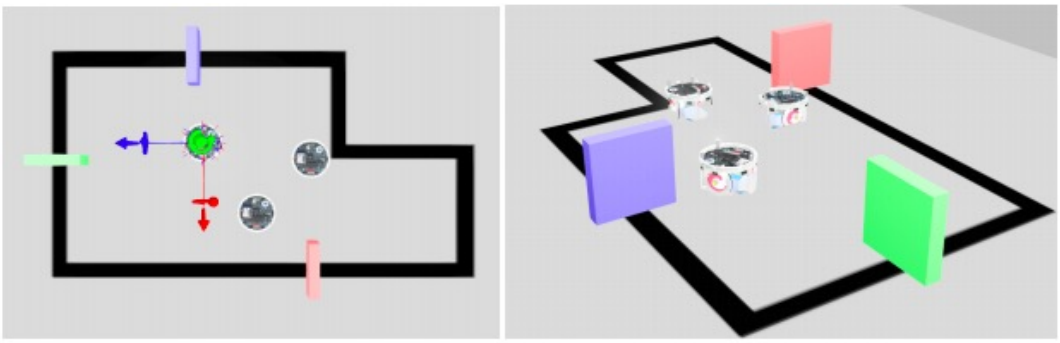

Figure 2.1.1: Project arena (closed line with a possible block configuration).

In phase one the robots are inside the circuit and can perform a LOVER or EXPLORER behavior. After finding a line, the robot must position itself in such a way that it follows the track counterclockwise. After the epuck has set up correctly, it will have to slalom red and blue obstacles if they appear in its path. The red obstacle has to be turned by keeping it on the left side of the robot and the blue one on the right side. The second phase ends when the robot encounters the green obstacle or another e-puck. In phase three the e-puck that ran into the green obstacle or another epuck will stand still until all the agents of the experiment have also stopped. At the end of this stage all obstacles will be removed and the robots have to start again following the track. The green obstacle will then be repositioned and phase three will be

repeated. When all the agents are stationary (in front of the green obstacle or behind another epuck) the experiment will be considered as finished. At each stage, a particular combination of LEDs is attributed to each behaviour (for more information on LED combinations please refer to Appendix B).

2.2 Solution Strategy

The clarity of the problem description has facilitated both the development of the strategy and its implementation. The strategy followed step by step the phases previously mentioned.

2.2.1 Line Follower

The robot could start its behavior either in LOVER or EXPLORER. The correct and more robust choice was the EXPLORER because it allows the robot to avoid obstacles before meeting the line. In the video of the 2.4 one can notice the dodging of the green obstacle before meeting the line.

At the beginning the e-puck must turn around the circuit counterclockwise after meeting a line. To follow the line the e-puck has three ground sensors, positioned at the same height.

A robust solution for following the lines requires two sensors (IR GROUND[1] and IR GROUND [2]) on the black line and one sensor (IR GROUND[0]) on the white line. Figure A.4 in Annex A.4 schematically illustrates the situation at equilibrium. Starting from inside the circuit the epuck will turn counterclockwise and adjust its speed to maintain this position. A table (2.1.) was created to detail the wheel speeds on a case by case basis in the event that the sensors pick up certain colours. In the notation of the first column there is the colour (W stands for white and B stands for black) picked up by each sensor. In the second and third column the speed of the left and right wheels is decided. A table similar to the one below was designed to define the positioning of the epuck from the outside, a discussion of this exception will be presented in 2.2.2.

Table 2.1: Line follower - speed regulation.

Sensor 012	**Left speed**	**Right speed**
NNN	0	NORM SPEED*3
NNW	-NORM SPEED	NORM SPEED*1.7
NWN	-NORM SPEED	NORM SPEED*1.7
NWW	-NORM SPEED	NORM SPEED*1.7
WNW	0	NORM SPEED
WWN	NORM SPEED*3	0
WNN	NORM SPEED	NORM SPEED
WWW	NORM SPEED*2	-NORM SPEED*2

A problem with this approach emerges together with the appearance of angles greater then 180deg in the circuit. In this specific case the robot will lose all three sensors at a stroke, switching to all sensors that pick up white. Given the direction of travel the concave corners will require the e-puck to always turn right. A robust solution is to rotate the e-puck 360 degrees before it enters the EXPLORER phase. This solution has proven effective in a series of tests in a world with irregular lines, see Figure A.6 in Annex A.4, even when the robot encounters angles of less than or greater than 270 degrees. A video showing a test can be seen by clicking on the following link :

https://youtu.be/nqPKA5VQxC4

2.2.2 Response in case of an obstacle

The color recognition of objects through the camera will be performed once the presence of an obstacle with the position sensors has been recognized. Only the central values of the captured camera image will be used to determine the color of the object. After the color has been classified the robot will stop in case of green or unidentified object (e-puck). If the recognized color is red or blue the agent will pass the obstacle with a PID controller. It is planned to use a counter for delaying the reactivation of the ground sensors and therefore to give the e-puck more time to perform the behavior. When the identified object is blue, the robot will go out of the circuit for a moment and come back through the outside. However, repositioning directly the e-puck may lead to a different result than originally intended. In fact the same set of sensors could lead the epuck to position itself outside the line and turn clockwise. Two functions will be used to exclude this second possibility. The first one will make sure that the robot is on the line and the second one will force it to turn right in order to resume the rotation in the desired direction. Figure A.5 in Annexe A.4 shows the steps of this solution.

Another possible solution to follow the line inside was to use the ground sensor values directly to adjust the wheel speed differential. This second option seemed at first sight the most correct and showed great potential also to solve the problem of angles greater than 180 degrees directly. Unfortunately the results obtained did not pass the test of the testing world environment (see Figure 2.2.2). For these reasons, the solution described previously was chosen.

2.2.3 Communication, loop and LEDs

The e-pucks will be constantly listening for receiving messages from other robots. It is planned to use only one communication channel that processes information on the current status of the other e-pucks and the number of rounds travelled. At the end a LED choreography will be performed that will alternate red LEDs to RGB LEDs.

2.3 Implementation

For space reasons most of the functions and variables mentioned in the text will not be immediately reported in this chapter. The library of all the functions used is accessible in Appendix A5. These functions are sorted alphabetically. In section A6 it will be possible to access the ROBOT LOOP() function and the MAIN() as well.

Before starting to program the behavior it is necessary to determine the values of the external objects. This was done by scanning the objects in a static and optimal situation.

Figure 2.3.1: The distance between the e-puck and the obstacle recorded.

Table 2.2. summarises the values for each external element whose value has been recorded by the sensors.

Table 2.2: Value of external elements.

Type of sensor	Object	Value
Proximity sensor	Obstacle	bigger then 100
Camera	Red	RGB(135,45,50)
Camera	Blue	RGB(75,70,180)
Camera	Green	RGB(80,150,60)
Ground sensor	Black	smaller then 550
Ground sensor	White	bigger then 550

The implementation makes use of several variables to recognize where the e-puck is located in the process. In the box below is a list of those functional to the whole process.

```
int phaseone = 1;
int object = 0;
int line =0;
int camera =0;
int counter=0;
int linegoodposition=0;
int on_the_line=0;
short int waiting_bots = 0;
int times=0;
```

The "line" and "linegoodposition" variables assume value 1 when a line is respectively recognized or when the desired position has been reached (in this case when the sensors have a WNN value).
If the epuck is not in position and the last recognized object was blue then the e-puck must

be positioned from outside the circuit. In this special case, it must be ensured that the e-puck has entered the line (here the use of the "linefollow entering" function) and has turned in the correct direction before inheriting the classic function that allows it to follow the circuit (called "linefollow position inside"). The specific functions are presented in Appendix A5.
Phase one ends (int phaseone=0) when an obstacle is recognized. The robot then turns on the camera to record the values (the function called is GET CAMERAXL).
If the recognized color is red (GET CAMERAXL == RED) the e-puck will execute the "PID RIGHT" function (K=0.12 and D=0.05) bypassing the obstacle clockwise. If the obstacle is blue (GET CAMERAXL == BLUE), the "PID LEFT" function (K=0.12 and D=0.05) will be called and the direction of rotation with respect to the obstacle will be counterclockwise.
For both rotations a counter of 100 iterations has been fixed before reactivating the position sensors, in order to give the epuck time to finish the behaviour.
In case of GREEN obstacles (GET CAMERAXL == GREEN) or stopped EPUCKs (GET CAMERAXL == EPUCK) a message ("rdy") will be sent and the variables "waitingbots" and "times" will be increased.
The e-puck will recognize that all robots are stopped when it receives the message ("rdy") from all other e-pucks. At the end of the first round each robot will wait for the obstacle function to give a zero result. If the e-puck is in front of a green obstacle, it will immediately restart to follow the line as soon as the obstacle in front of it is removed. If the e-puck is behind and the latter is back in motion then the former will wait 40 steps before moving forward. This will allow each robot to travel with greater freedom.
At the end of the second round the e-pucks will alternate the red LEDs to the green RGB LEDs to tell the operator that all the agents are stationary and the behavior has been completed.

2.4 Validation

The solution presented makes exclusive use of IF statement in the controller. All functions (EXPLORER, LINE FOLLOW POSITION INSIDE, GET CAMERA XL, etc.) have been kept separate in another library and are accessible in Appendix A5. This has made programming easier and the final solution more readable. This implementation allows to solve the problem in a high performance way, leaving also room for an external operator to make changes in the environment such as choosing the number of loops and e-pucks. There were some difficulties during implementation in getting the robot to follow the line when changing corners. All these difficulties were successfully overcome. A video exposing and explaining step by step the various stages can be viewed by clicking the following link

https://youtu.be/uv080kCx4$_8$

Chapter 3

Challenge

This chapter is dedicated to the implementation of another behavior. The problem statement was chosen by a team of three students from the University of Fribourg (M. Krishna LeMoing, M. Pietro Bonazzi and M. Till Valentin Aebi). The structure of this chapter is identical to the previous one. After the presentation of the problem (3.1), a description of the strategy will follow. (3.2). Finally the implementation of the solution will be explained (3.3) and then the quality of the final solution will be discussed (3.4).

3.1 Problem statement

An arena created in webots contains a maze of physical walls. The floor of the arena consists of a series of squares delimited by black lines. Each wall is positioned at the center of at least one side of a square and perpendicular to the line itself.

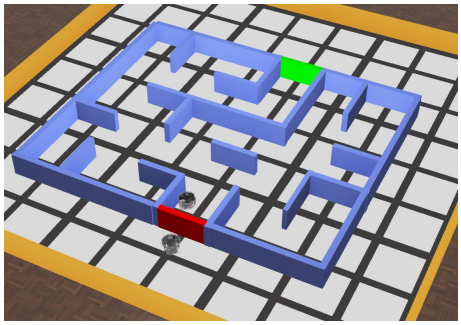

Figure 3.1.1: The challenge's world.

The first e-puck scans the entire maze and sends the best path to the second e-puck. The second e-puck simply has to follow the instructions to find the exit immediately. An external operator must be able to change the size of the maze at will and choose the xy coordinates of the beginning and end of the maze without affecting the controller.

3.2 Solution Strategy

The strategy implemented foresees the creation in separate modules of the various stages necessary to solve the problem. Testing will mostly be carried out in another smaller and simpler labyrinth so as to save time and facilitate debugging. The solution will then be made scalable to fit any version of the maze that meets the conditions set out in the problem statement.

3.2.1 Mapping

The first robot (ID=1) must first scan the entire maze. To do this the robot will follow the lines on the ground (using the ground sensors). When an intersection (or a node) has not yet been registered, the e-puck will have to turn, advance and go back from all four corners. The approach is necessary to recognize (using the proximity sensors) any walls positioned in the middle of the square.

Data on the presence of walls is stored in a structure called a node. Each node has x and y coordinates that differentiate it from the others. In addition, it contains a number of variables. The first element of the structure is the wall[] matrix. In it there is recorded the presence (0) or the absence (1) of walls on the four sides of the cross. The second element of the structure relevant for this phase is the checked variable set to 1 when the node has already been scanned (and to 0 by default at the beginning). The scanner phase will be performed for all the nodes of the maze. The recording of the walls will thus update the variables "wall[]" and "checked" in each node of the already created computer's copy (our maze) of the physical maze.

3.2.2 Dijkstra's algorithm and relative path

Another variable of the node structure gives the reference regarding the distance to the end (set to 32767 by default at the beginning). After scanning a function will be called that will compare the distances of the various nodes from the end of the maze. Starting from the end (distance=0), a unit will be added for each side of the node that does not contain a wall (and therefore reachable by an e-puck). This function will be recursive and will be performed for each accessible side by entering the shortest distance for each node from the end.

The principle to obtain this solution is inspired by Dijkstra's algorithm. If the distance already recorded in the side of the node where one wants to access is shorter than the one that could be entered then the function will not change any parameter. Otherwise the distance parameter will be updated and the direction of access of the shortest route towards the end will be entered. The direction data is initially absolute and will be changed to relative in another function. This function will transform the direction data and record the instructions in the matrix to be sent to the second robot. The box below summarizes the structure of a node. The last important data of the structure is the special variable which is set to one for the node containing the beginning, to two for the node containing the end and to zero for all other cases.

```
struct maze_node {
    short int walls[4];
    short int distance; //from the end
    short int direction; //of the next node in the shortest path to the end
    short int special; //marking if a node is normal: 0, start: 1, end: 2
    short int checked; //once the walls of a node have been scanned, this variable will be made 1 to mark it as
        checked
};
```

3.2.3 Communication and execution

The path[] matrix containing the solution is nothing more than a series of numbers that give relative indications to execute the shortest path.

```
#define LEFT 0
#define FORWARD 1
#define RIGHT 2
#define BACK 3
#define NONE 5
#define FINISH 4
```

Since the communication's channel accepts only characters then each number will be converted into characters first, then sent and finally converted back into a number.

3.3 Implementation

For space reasons most of the functions and variables mentioned in the text will not be immediately reported in this chapter. The library of all the functions used is accessible in Appendix B1. These functions are sorted alphabetically. In section B2 it will be possible to access the ROBOT LOOP() function and the MAIN() as well. The first function called in the controller,

newmaze(), creates the computer copy of the maze using operator-defined inputs. The newnode() pointer function creates and initializes each node with standard variables. For example, the distance of each node from the end is set to 32767 (the maximum for a small integer). This data will then be used to calculate the optimal distance.
The controller's subsequent initial instructions set the coordinates of the initial node of the maze, the first direction of travel to be executed, the special characters at the beginning and end of the maze, and the modes of the two epucks. The mode of the first robot is SCAN. In SCAN the called function is checkwalls(). In checkwalls() the robot will go back and forth for each side of the cross checking for walls at the ends using the scanwalls() function.
When registration has been made for each side, the robot switches to the TURN phase. The function called here is turn(). In this function the robot finds the direction in which to proceed to the next node thanks to the findpath() function. The variable currentdir is equal to 0 at the beginning. The robot will use the module operator to first see if there is a wall to its left. If there is no wall (wall[3]==1) then the e-puck will exit from the first node on the left. If not then the e-puck will look forward, then right and finally back to find a free side to progress from.
Once the direction is found, the e-puck will use this information to return to the correct position and then follow the line in DRIVE mode. DRIVE mode calls the follow line() function. During this function the robot follows the line and updates the coordinates to change the reference to the new node it is heading to.
When all three sensors pick up the black line then the robot recognizes that it has finally arrived at the new node. Here the robot has three possibilities. The first option concludes the scanning phase of the robot with ID=1. In fact if the node where the e-puck is located contains the beginning of the maze, the e-puck itself will turn on its left, stop, calculate the shortest path and send it to the second robot.
The second option regulates cases where the node has already been checked. In this case the epuck will directly repeat the TURN mode without having to scan the node a second time. The third and last possibility concerns a node not yet scanned. The robot will then return to SCAN mode. When the epuck returns to the initial node it should have scanned the whole maze. The function called in this situation is finish program(). In this function is immediately called the findpath() function which in turn calls two functions: checknode(int node x, int node y, int distance, int direction) and relative path().
The checknode function has the following initial inputs : (ENDX, ENDY, 0, SOUTH). The purpose of the function is to assign the minimum distance of each node from the end of the maze and reverse the direction of travel used in the analysis so that the references are in line with a robot starting from the beginning. The initial data for the distance factor of each node is 32767. The first distance input is zero. An if statement will then check if zero is less than 32767. Since this is true the distance of the starting node will become the new distance (i.e. 0). As previously mentioned, using a module operator, the direction of the analysis will be reversed

giving the opposite direction. In this way the absolute direction corresponds to the one found in the labyrinth starting from the beginning. The SOUTH input thus becomes NORTH.

```
#define NORTH 0
#define EAST 1
#define SOUTH 2
#define WEST 3
```

```
        current_node->direction = (direction + 2) % 4;
```

In the last part, the checknode() function will call itself recursively for all the wall-free ends of the new node and increase the input distance of a unit. The latter in turn will call the accessible nodes from their position and in a short time the absolute distances and directions to each node will be known.
If the algorithm finds directions less than the distance that can be entered from a given node then the above procedure will ignore the second possibility and leave the distance and direction data intact.
The second robot however needs the relative direction with respect to the node in which it is positioned (STARTX, STARTY). To set the actual instructions the relative path() function is used. This function uses the data on the direction recorded in the node (NORTH,SOUTH,EAST,WEST) and compares it with that of the previous node to indicate the actual direction that the robot must execute (FORWARD,RIGHT,LEFT,FINISH).

```
#define LEFT 0
#define FORWARD 1
#define RIGHT 2
#define BACK 3
#define NONE 5
#define FINISH 4
```

When the above function has changed the directions of the optimal path leading to the end (special==END), the loop is exited and the last instruction (FINISH) is recorded in the path[] matrix. This set of instructions will be nothing more than a series of integers that must be converted into characters, using the tochar() function, to be transmitted in the communication channel. Finally, the second robot will receive the matrix, transform it back into whole numbers using the toint() function and execute the optimal path from start to finish.

3.4 Validation

The solution obtained reflects expectations. It was tested in two different labyrinths and no problems were found. During the implementation the difficulties were overcome with creativity and success. The video below per minute illustrates the solution obtained. It is accompanied by a short audio description of the various situations.

https://youtu.be/uv080kCx4$_8$

Chapter 4

Conclusion

In this project two different behaviors were implemented on an e-puck 2 robot and simulated in webots. The coordination of the different sensors was successfully and effectively achieved. In the first controller the robots can also follow hand drawn lines with extreme angles. It has been tested that the second controller would also work with other mazes. From the very beginning, the transferability of code has been one of the key points in behavior programming. In the future it would be interesting to find an equation that would allow to follow the lines on the ground using directly the values detected by the sensors and by directly bypassing the speed differential. It would also be equally interesting to make the camera capable of analyzing objects. One of the flaws of the maze controller is that it may not explore every node if there are particular loops. So in the future this problem could be addressed or other features could be added to the controller itself to make it more complete. It would be interesting to allow the controller to find the solution in case the end of the maze is not known. In addition, it could be useful to manage the case in which a robot in SCAN phase unloads or breaks in the middle of the maze scanner, before sending it, in order to allow a replacement to avoid having to do the exploration all over again.

Bibliography

[1] Justin Zobel. *Writing for Computer Science*, 2nd edition. Springer-Verlag, London, 2004, 275 pages.

[2] Valentino Braitenberg. *Vehicles: Experiments in Synthetic Psychology*. MIT Press, 1986.

[3] *Webots Reference Manual*. `https://www.cyberbotics.com/reference.pdf` version 2019a Last visited: 11.02.2019.

Appendix

Appendix A Task

A.1 Problem Statement

The entire problem statement can be found in the next page.

Department of Informatics | Faculty of Science | University of Fribourg | Switzerland

Project description and guidelines

IN 2022 Robotics, BSc Course, 2nd Sem., Dr. Julien Nembrini, Valeria Schmidt

Handout on Thursday April 2 2020 Week 7 to 13 Due on **Sunday May 24 2020, midnight**

Presentation on **Thursday May 28 2020**

The remaining part of the semester will be devoted to a group project which consists of a basic task and a challenge. The challenge can be freely defined by the students but must be presented to the teaching team for approval before starting work on it.

Important dates :

April 8, 23h00 Short description of challenge idea, sent per email to the teaching team

April 9 Challenge idea discussion

April 20 Final short challenge description, sent per email

May 24, midnight Hard deadline for submitting the final project (incl. presentation)

May 28, 13h15 Final presentation

The techniques exercised during the first part of the semester are sufficient in combination to solve the task proposed. All behaviors needed are variants of Explorer/Lover behaviors or PID controllers. However, it is allowed to use other techniques if wished. As during the first part of the semester, all robots should use the exact same code (unless especially agreed for the challenge).

Grade The final project is evaluated on the basis of :

- the project code/algorithm (30%)
- the challenge code/algorithm (20%)
- the report (30%)
- the video (10%)
- the final presentation (10%)

Important elements include :

- dynamicity, robustness and efficiency of the algorithm
- clarity and readability of the code
- structure and clarity of the report
- orthography and grammar

Hand in :

Upload your code, report, video and presentation (pdf) on Moodle before May 24, (midnight). You may amend your presentation between the deadline and May 28, but in this case **you must resubmit it per email before 12h00 on May 28**.

Zip files not following the naming convention will not be considered.

Department of Informatics | Faculty of Science | University of Fribourg | Switzerland

Basic task

The basic task consists of letting 3 robots (1) find the closed line on the ground and follow it anticlockwise (2) slalom around colored blocks depending on their color (red → block on the robot's right side, blue → block on the robot's left side) or stop at the green block and gather (3) loop around the line without blocks together and stop at the green block again.

Environment

In the basic task, the robot arena, through which the e-pucks navigate, has a height and width of 80x160 centimeters. The robots start in the interior of a closed line on the ground. Colored blocks are positioned on the line (see fig 1). Apart from the fact that these blocks cut the line perpendicularly, the configuration of the blocks along the line is not fixed : the algorithm should be robust to blocks positioned differently and with colors distributed differently over the line. Note that there can be only one green block and that the blocks will not be placed near the line corners.

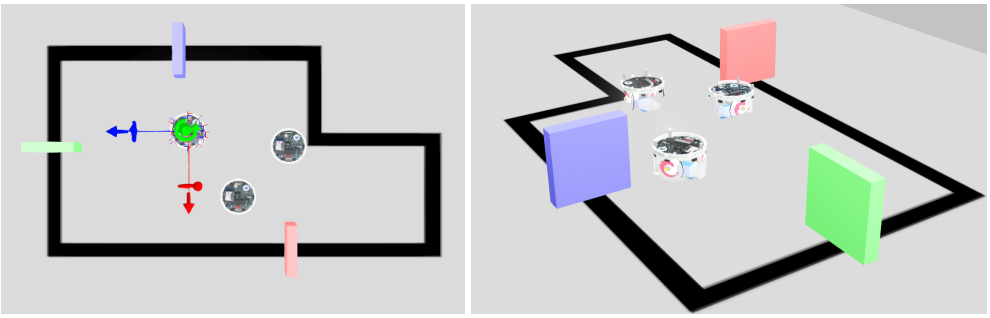

Figure 1 – Project arena (closed line with a possible block configuration)

In general, robots are not allowed to leave the line on the ground, except when they go around blocks.

We assume that 3 robots are in the arena and that the communication is reliable. Each robot is intially placed randomly within the area enclosed by the line. Each robot then follows the following phases (in order) :

- Phase 1 : Find the line and follow it anticlockwise.
- Phase 2 : Detect blocks in front and go around them according to their color, or stop at the green block.
- Phase 3 : Together, loop around the line without blocks and stop again at the green block (replaced).

Phase 1, Find the line

Task to achieve : find the line and follow it anticlockwise.

1. Each robot finds the line and follows it anticlockwise while robustly avoiding blocks and other robots.
2. Once a robot has detected the line, phase 2 starts.

Phase 2, Slalom

Task to achieve : each robot go around obstacles differently depending on their color, or stop at a green block (or behind a stopped robot).

1. Each robot robustly goes around blocks present on the line depending on the blocks' color.
2. Red blocks have to be on the robot's right side ; blue blocks on the robot's left side
3. The first robot encountering a green block must stop in front of it
4. Once all robots are stopped at the green block or behind a robot next to it, phase 3 starts.

Phase 3, Loop

Task to achieve : Loop around the closed line while staying together. Stop all robots at the green block again.

When all robots have gathered at the green bloc. All blocks are manually removed and all robots move around the closed line while staying together. The green block is replaced on the line and robots must stop again when reaching it (either in front of it or behind a robot). Robots must collectively and robustly decide that the task is completed and signal task completion with the appropriate LED pattern.

Challenge

The challenge consists of defining, designing and solving an additional challenging task.

The challenge can be freely defined by the groups, but must be described in a short email (5 to 6 sentences at most) that will be sent before **April 8 midnight**. This idea will be discussed with the teaching team on **April 9** to get approval before starting work on it. A final short description will be sent per email before **April 20, midnight**. Given the simulation only context, project involving a large number of robots are encouraged. Possible challenges are :

- autonomous "cars"
- collectively solve a maze
- recognize objects/forms
- neural network to improve robot behaviour
- make multi-robot formations
- **your proposal**

Each group will have to choose a different challenge. If two groups choose something similar, variants that are sufficiently different will have to be proposed. If you are hesitating between multiple ideas, feel free to include all of them in your initial mail.

Department of Informatics | Faculty of Science | University of Fribourg | Switzerland

Guidelines

You are free to choose which strategy you implement for solving the basic task and the challenge, but keep the following guidelines in mind :

- You are expected to discuss the problem, including coding and testing, in group of three persons. Please inform if you wish to change group before starting the project.

- **Report :** each student has to write his own report in Latex, maximum 12 pages + appendices (including this document as Appendix A). The report should present the project (basic task + challenge) as a whole. An empty template is provided on moodle
- **Code :** one code per group. We ask you to hand in a modular, readable and commented code. Your code will be thoroughly read.
- **Video :** one video per group, max 4 minutes. An uninterrupted and accelerated shot of the whole basic task process (phases 1 to 3) must be included. Any acceleration must be annotated in the video and further annotations for clarity are encouraged.
- **Presentation :** each student has to do his own presentation (max. 4 minutes) focusing on a particularly interesting aspect of the project (task or challenge), different for each student.

- The e-pucks need to collaborate and communicate. 3 e-pucks have to be used for the task.
- Navigating the environment can be done using the lover/explorer behaviors and PID controller (and possibly others). Be careful to the robustness of other controllers you may choose to use.
- Don't use any hard coded behavior. The e-pucks should be able to complete the basic task even if their starting positions change or a different arena is used (all the robots basically know is that the arena is divided by a closed line within which they start and that colored blocks are disposed along this line).
- Make your solution as dynamic (independent of initial configuration), robust (catch error situations) and efficient (achieving the task in a short time) as possible.
- Document your code (events, methods, algorithms, etc.).
- Use the LED patterns described in the appendix in order to indicate to an observer what the robots are doing. If you add custom patterns, document it using the same tabular presentation as in the appendix.

Appendix : LED patterns

The following is a list of the LED patterns that must be used. For each pattern, the indexes of the LEDs are given. The LEDs have to be turned on long enough to be visible in the video.

Behaviours		LED indexes	Comment
Recognized colors :	Red	all RGB 0xff0000	
	Green	all RGB 0x00ff00	
	Blue	all RGB 0x00ffff	
General behaviours :	Explorer	1	
	Lover	2	
	Line following	0	
	Wall following	0,1,2/0,2,3	side dependent
Phase 2 completed	Line following	1,2,3	
Phase 3 completed	In front of green block	0,1,3	
	In front of robot	1,3	
Task completed		ad-lib	choose your favorite !
Others			to be specified by the students

A.2 LED's Recognition

The image below shows the arrangement of the LEDs in an e-puck-2.

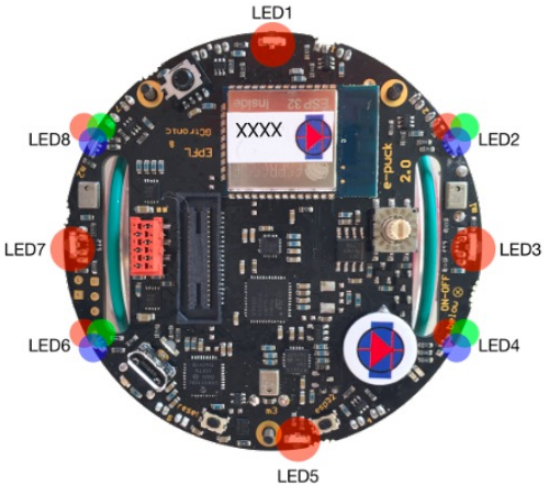

Figure A.1: LED disposition on the e-puck2

The following is a list of the LED patterns that must be used. For each pattern, the indexes of the LEDs are given. The LEDs have to be turned on long enough to be visible in the video.

Behaviours		LED indexes	Comment
Recognized colors :	Red	all RGB 0xff0000	
	Green	all RGB 0x00ff00	
	Blue	all RGB 0x00ffff	
General behaviours :	Explorer	1	
	Lover	2	
	Line following	0	
	Wall following	0,1,2/0,2,3	side dependent
Phase 2 completed	Line following	1,2,3	
Phase 3 completed	In front of green block	0,1,3	
	In front of robot	1,3	
Task completed		ad-lib	choose your favorite!
Others			to be specified by the students

Figure A.2: LED behavior

A.3 Images

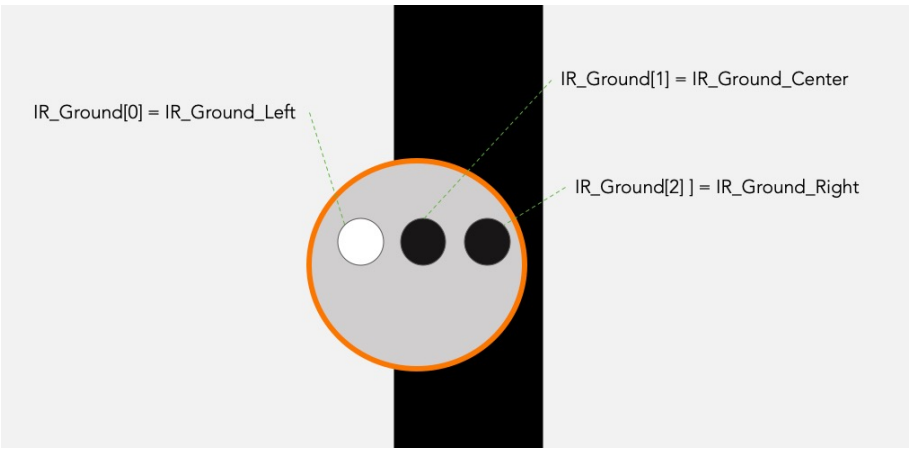

Figure A.3: The idea behind the line following.

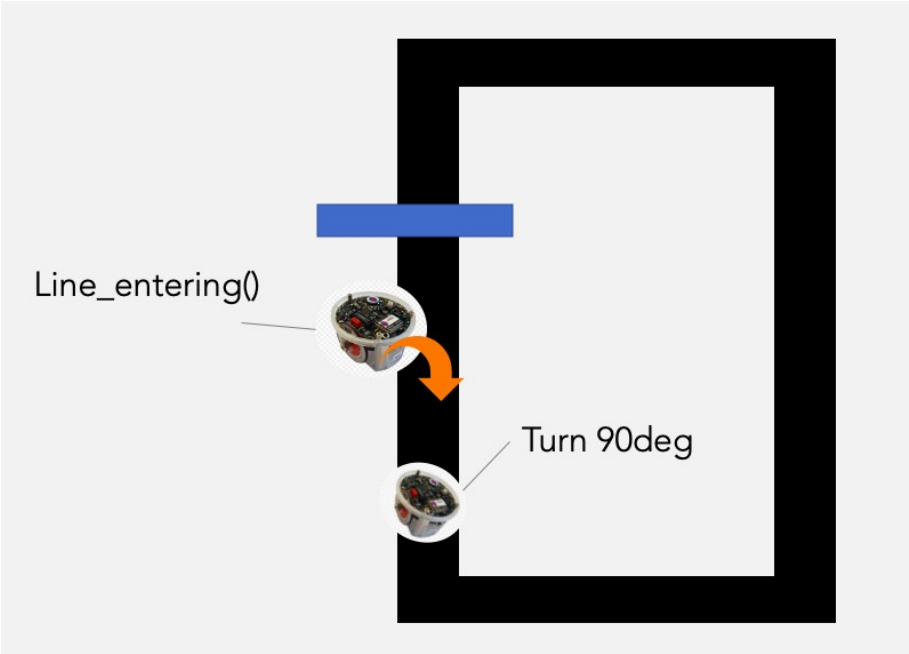

Figure A.4: Entering from the outside.

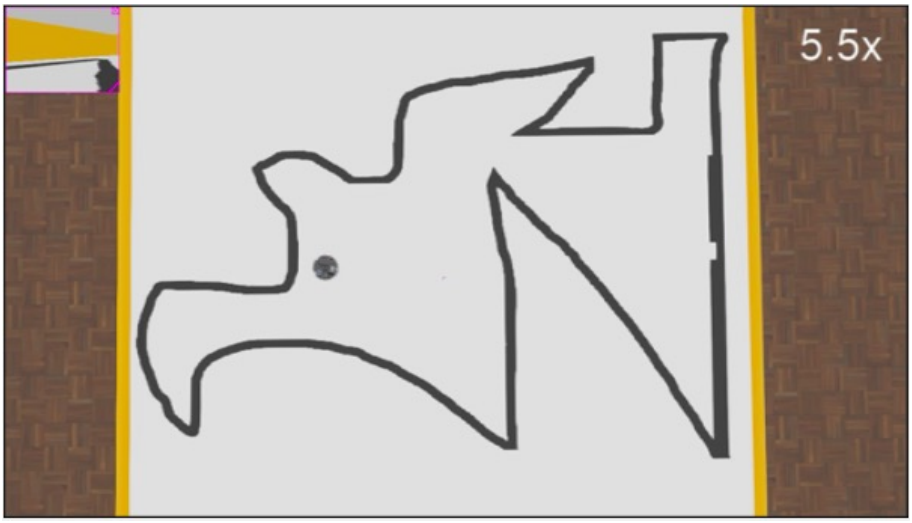

Figure A.5: The testing world.

A.4 Functions

The EXPLORER function :

```
void explorer() {
  led_reset();
  enable_led(1);
    short int prox_values[8];
  get_prox(prox_values);

        double prox_left = (4 * prox_values[7] + 2 * prox_values[6] + prox_values[5]) / 7;
        double prox_right = (4 * prox_values[0] + 2 * prox_values[1] + prox_values[2]) / 7;

        double ds_left = (NORM_SPEED * prox_left) / MAX_PROX;
        double ds_right = (NORM_SPEED * prox_right) / MAX_PROX;

        double speed_left = bounded_speed(NORM_SPEED - ds_right);
        double speed_right = bounded_speed(NORM_SPEED - ds_left);
        set_speed(speed_left, speed_right);
}
```

The FORWARD function :

```
void forward() {
        set_speed(NORM_SPEED, NORM_SPEED);
}
```

The GET CAMERAXL function :

```
int get_cameraXL(){
    led_reset();
    unsigned char red[CAMERA_WIDTH * CAMERA_HEIGHT];
    unsigned char green[CAMERA_WIDTH * CAMERA_HEIGHT];
    unsigned char blue[CAMERA_WIDTH * CAMERA_HEIGHT];
    get_camera(red, green, blue);

    int redpixel = 0;
    int greenpixel = 0;
    int bluepixel = 0;
    int redcount = 0;
    int greencount = 0;
    int bluecount = 0;
    int othercount =0;

    for (int n=55; n<64; n++) {
        for (int m=0;m<CAMERA_WIDTH;m++) {
        redpixel = red[n*CAMERA_WIDTH + m];
        greenpixel = green[n*CAMERA_WIDTH + m];
        bluepixel = blue[n*CAMERA_WIDTH + m];

            if (redpixel >  150 && greenpixel <  80 && bluepixel < 80)
            {redcount++;}

            else if (redpixel <  80 && greenpixel >  120 && bluepixel < 70)
            {greencount++;}

            else if (redpixel <  85 && greenpixel <  85 && bluepixel > 150)
            {bluecount++;}

            else {othercount++;}
        }
    }

    if((bluecount>redcount) && (bluecount>greencount) && (bluecount>othercount))
    {end = BLUE;
    for (int i = 0; i < 4; i++)
    {enable_rgbled(i,0x00fff);}}

    if((greencount>redcount) && (greencount>bluecount) && (greencount>othercount))
    {end = GREEN;
    for (int i = 0; i < 4; i++)
    {enable_rgbled(i,0x00ff00);}}

    if((redcount>greencount) && (redcount>bluecount) && (redcount>othercount))
    {end = RED;
    for (int i = 0; i < 4; i++)
    {enable_rgbled(i,0xff0000);}}

    if((othercount>redcount) && (othercount>bluecount) && (othercount>greencount))
    {end = EPUCK;}

    return end;
}
```

The LED RESET function :

```
void led_reset() {
    for (int i = 0; i < 4; i++)
    {disable_led(i);}
    for (int n = 0; n < 4; n++)
    {disable_rgbled(n);}
}
```

The LINEFOLLOW DETECTED function :

```
int linefollow_detected () {
        short int IR_ground[GROUND_SENSORS_COUNT];
        get_ground(IR_ground);
            if (IR_ground[0] > LINE_STEP && IR_ground[1] > LINE_STEP && IR_ground[2] > LINE_STEP)
              {return 0;}
            else
              {return 1;}
}
```

The LINEFOLLOW ENTERING function :

```
void linefollow_entering() {

        short int IR_ground[GROUND_SENSORS_COUNT];
        get_ground(IR_ground);

        double IR_ground_left = IR_ground[0];
        double IR_ground_center = IR_ground[1];
        double IR_ground_right = IR_ground[2];


         if (IR_ground_left < LINE_STEP && IR_ground_center < LINE_STEP && IR_ground_right < LINE_STEP) {
             forward();
             printf("NNN\n");
         }
        if (IR_ground_left < LINE_STEP && IR_ground_center < LINE_STEP && IR_ground_right > LINE_STEP) {
             set_speed((IR_ground_left/LINE_STEP), (IR_ground_right/LINE_STEP));
             printf("NNW\n");
         }
        if (IR_ground_left < LINE_STEP && IR_ground_center > LINE_STEP && IR_ground_right < LINE_STEP) {
             set_speed(NORM_SPEED, NORM_SPEED);
             printf("NWN\n");
         }
        if (IR_ground_left < LINE_STEP && IR_ground_center > LINE_STEP && IR_ground_right > LINE_STEP) {
             set_speed((IR_ground_left/LINE_STEP), (IR_ground_right/LINE_STEP));
             printf("NWW\n");
         }
        if (IR_ground_left > LINE_STEP && IR_ground_center < LINE_STEP && IR_ground_right > LINE_STEP) {
             set_speed(NORM_SPEED, NORM_SPEED);
             printf("WNW\n");
         }
        if (IR_ground_left > LINE_STEP && IR_ground_center > LINE_STEP && IR_ground_right < LINE_STEP) {
             set_speed((IR_ground_left/LINE_STEP), (IR_ground_right/LINE_STEP));
             printf("WWN\n");
         }
}
```

The LINEFOLLOW GOODPOSITION function :

```
int linefollow_goodposition() {
        short int IR_ground[GROUND_SENSORS_COUNT];
        get_ground(IR_ground);
        if ((IR_ground[0] > LINE_STEP && IR_ground[1] < LINE_STEP )&& IR_ground[2] < LINE_STEP)
        { set_speed(NORM_SPEED, NORM_SPEED);
          return 1;}
        else
        {return 0;}
}
```

The LINEFOLLOW ONTHELINE function :

```
int linefollow_ontheline() {
        short int IR_ground[GROUND_SENSORS_COUNT];
        get_ground(IR_ground);
        if ((IR_ground[0] < LINE_STEP && IR_ground[1] < LINE_STEP )&& IR_ground[2] < LINE_STEP)
        {return 1;}
        else
        {return 0;}

}
```

The LINEFOLLOW POSITION INSIDE function :

```
void linefollow_position_inside() {

        short int IR_ground[GROUND_SENSORS_COUNT];
        get_ground(IR_ground);

        double IR_ground_left = IR_ground[0];
        double IR_ground_center = IR_ground[1];
        double IR_ground_right = IR_ground[2];


         if (IR_ground_left < LINE_STEP && IR_ground_center < LINE_STEP && IR_ground_right < LINE_STEP) {
             set_speed(0, NORM_SPEED*3);
             printf("NNN\n");
         }
        if (IR_ground_left < LINE_STEP && IR_ground_center < LINE_STEP && IR_ground_right > LINE_STEP) {
             set_speed(-NORM_SPEED, NORM_SPEED* 1.7);
             printf("NNW\n");
         }
        if (IR_ground_left < LINE_STEP && IR_ground_center > LINE_STEP && IR_ground_right < LINE_STEP) {
             set_speed(-NORM_SPEED, NORM_SPEED* 1.7);
             printf("NWN\n");
         }
        if (IR_ground_left < LINE_STEP && IR_ground_center > LINE_STEP && IR_ground_right > LINE_STEP) {
             set_speed(-NORM_SPEED, NORM_SPEED* 1.7);
             printf("NWW\n");
         }
        if (IR_ground_left > LINE_STEP && IR_ground_center < LINE_STEP && IR_ground_right > LINE_STEP) {
             set_speed(0, NORM_SPEED);
             printf("WNW\n");
         }
        if (IR_ground_left > LINE_STEP && IR_ground_center > LINE_STEP && IR_ground_right < LINE_STEP) {
             set_speed(NORM_SPEED*3, 0);
             printf("WWN\n");
         }
}
```

The OBSTACLE function :

```
int obstacle () {
    short int prox_values[8];
    get_prox(prox_values);
    int prox_right_front = prox_values[0];
    int prox_left_front = prox_values[7];
    double prox_front_right = (0.5 * prox_values[0] + 0.5 * prox_values[1]);
    double prox_front_left = (0.5 * prox_values[7] + 0.5 * prox_values[6]);

    if (prox_left_front > 90 || prox_right_front > 90 || prox_front_right>100 || prox_front_left>100)
    {return 1 ;}
    else
    {return 0;}
}
```

Values for the PID :

```
#define TIME_STEP 64
#define Ka 0.12
#define Kb 0.12
#define T_I 0
#define T_D 0.05
#define PID_WALL_FOLLOW_TARGET 80
#define PID_MAX_DS 100

#define a 0.35
#define b 0.4
#define c 0.25
#define d 0
```

The PID LEFT function :

```
void PID_LEFT() {

short int prox_values[8];
 get_prox(prox_values);
  double prox_left = (a * prox_values[7] + b * prox_values[6] + c * prox_values[5] + d * prox_values[4]) / (a+b+c
       +d);

  double prev_err = error;
  error = prox_left - PID_WALL_FOLLOW_TARGET;
  deriv = (error - prev_err)*1000/TIME_STEP;
  integ += error*TIME_STEP/1000;


  //double ds = Kb * error;
  double ds = Kb * ( error + T_D * deriv);
  //double ds = K * ( error + 1.0 / T_I * integ + T_D * deriv);
  double speed_right = (abs(ds)>PID_MAX_DS ? 0 : NORM_SPEED) - ds;
  double speed_left  = (abs(ds)>PID_MAX_DS ? 0 : NORM_SPEED) + ds;

  set_speed(bounded_speed(speed_left), bounded_speed(speed_right));


}
```

The PID RIGHT function:

```
void PID_RIGHT() {

short int prox_values[8];
 get_prox(prox_values);
 double prox_right = (a * prox_values[0] + b * prox_values[1] + c * prox_values[2] + d * prox_values[3]) / (a+b+c
      +d);

  double prev_err = error;
  error = prox_right - PID_WALL_FOLLOW_TARGET;
  deriv = (error - prev_err)*1000/TIME_STEP;
  integ += error*TIME_STEP/1000;


  //double ds = Ka * error;
  double ds = Ka * ( error + T_D * deriv);
  //double ds = K * ( error + 1.0 / T_I * integ + T_D * deriv);
  double speed_right = (abs(ds)>PID_MAX_DS ? 0 : NORM_SPEED) + ds;
  double speed_left  = (abs(ds)>PID_MAX_DS ? 0 : NORM_SPEED) - ds;

  set_speed(bounded_speed(speed_left), bounded_speed(speed_right));

}
```

The ROBOT SETUP function :

```
void robot_setup() {
    init_robot();
    init_sensors();
    init_camera();
    init_communication();
}
```

A.5 Controller

Controller's code for Chapter 2 :

```
void robot_loop() {

    int phaseone = 1;
    int object = 0;
    int line =0;
    int camera =0;
    int counter=0;
    int linegoodposition=0;
    int on_the_line=0;
    short int waiting_bots = 0;
    int times=0;

    while (robot_go_on()) {

    //the robots are always listening
    char rcv[MSG_LENGTH];
    char tmp[MSG_LENGTH];
    receive_msg(rcv);
    sprintf(tmp, "rdy");
    if (strcmp(rcv, tmp)==0)
        {waiting_bots++;
        times++;}


//Phase 1 : Find the line and follow it anticlockwise.
    if (phaseone>0){
        led_reset();
        line=linefollow_detected(); // checks if their if the robot detects any part of a line
        if (line<1){
            if (counter<70){ // adjustment before the explorer mode for angles greater then 180deg
                set_speed(NORM_SPEED*2,-NORM_SPEED*2);
                counter++;
            }
            else{
                explorer();    // searches for a line in explorer function
                led_reset();
                enable_led(1);
            }
        }
        else{
            led_reset();
            enable_led(0);
            linegoodposition=linefollow_goodposition();
            if (linegoodposition<1){     //the robot is not in the correct position
                if (camera==BLUE) {   // supplementary response when the robot comes from the outside
                    on_the_line=linefollow_ontheline(); // checks if it is on top of a line
                    if (on_the_line<1){    // if it isn't on top of a line it will enter the line
                        linefollow_entering();}
                    else{ // if it is on top, it will reposition anticlockwise
                    set_speed(NORM_SPEED,-NORM_SPEED);
                        if (counter<25){
                            counter++;}
                        else{
                            camera=0; // exit the supplementary response
                            counter=0;}
                    }
                }
                else {
                    linefollow_position_inside();}      //this will make the robot position itself --> WBB
            }

            else{ // the robot is positioned
                forward();
                camera=0;
                counter=0;
                led_reset();
                for (int i = 1; i < 4; i++)
                {enable_led(i);}
                object= obstacle(); // see if there is an obstacle on the way
                if (object>0)
                    {phaseone = 0;} // if there is, go to phase 2
            }
        }
    }

//Phase 2a : Detect blocks in front and go around them according to their color.

    else if (phaseone<1)
    {
            if (camera<1){
             // this function checks the color the camera will spot, it will assign it an int value so that we
                 will know which color is seen
                camera=get_cameraXL();
                // if the color is green or an epuck, the epuck will stop, send a message in the system and wait
                if (camera==GREEN || camera==EPUCK){
                        set_speed(0,0);
                        char tmp[MSG_LENGTH];
                        sprintf(tmp, "rdy");
```

```
                        send_msg(tmp);
                        waiting_bots++;
                        times++;
                }
                else{}
            }

            else{
                    // if the camera detects blue, it will go around it by the left.
                    if (camera==BLUE)
                    {
                            PID_LEFT();
                            if (counter<100){ //wait for the robot to complete the behavior
                                counter++;}
                            else{
                                line=linefollow_detected(); //start to detect lines again while in PID state
                                if (line>0) {// this will reset the phase, once it finds another line it will have to go back to the line following
                                  phaseone=1;
                                  counter=0;
                                }
                                else{}
                            }

                    }
                    else if
                        // if the camera is red it will go around by the right
                        (camera==RED){
                            PID_RIGHT();
                            if (counter<100){ //same as before
                                counter++;}
                            else{
                                line=linefollow_detected();
                                if (line>0) {//same as before
                                  phaseone=1;
                                  counter=0;
                                }
                                else{}
                            }

                    }

                    else if (camera==GREEN){   // this robot is the first to stop
                        led_reset();
                        enable_led(0);
                        enable_led(1);
                        enable_led(3);
                        // this will check the number of waiting bots, so that once they are all waiting it can continue
                        if (waiting_bots == NR_EPUCK) {
                            if (times== NR_TURN*NR_EPUCK){ // if the number of turns we want the epucks to do are completed
                                if (counter<10){ // then LED coreography
                                    led_reset();
                                    for (int i = 0; i < 4; i++)
                                    {enable_led(i);}
                                    counter++;
                                }
                                else{
                                    led_reset();
                                    for (int i = 0; i < 4; i++)
                                    {enable_rgbled(i,0x00ff00);}
                                    counter=0;
                                }
                            }

                            else{ // if the robot has not completed the wanted number of turns
                                object= obstacle(); //wait for the supervisor to remove the obstacle
                                if (object<1){
                                    phaseone = 1; //re follow the line
                                    waiting_bots = 0;
                                    camera =0;
                                    }
                                else{}
                            }
                        }
                        else{}
                    }

                    else if (camera==EPUCK){   //this robot is behind another bot
                        led_reset();
                        enable_led(1);
                        enable_led(3);
                        if (waiting_bots == NR_EPUCK){
                            if (times== NR_TURN*NR_EPUCK){
                                if (counter<10){
                                    led_reset();
                                    for (int i = 0; i < 4; i++)
                                    {enable_led(i);}
                                    counter++;
                                }
                                else{
                                    led_reset();
                                    for (int i = 0; i < 4; i++)
                                    {enable_rgbled(i,0x00ff00);}
```

```
                                    counter=0;
                                }
                            }
                            else{
                                object= obstacle();
                                //this will create a little delay in between the departure of the e-pucks so that they don't all go at the same time
                                if (object<1){
                                    if (counter<40)
                                        {counter++;}
                                    else{
                                        counter=0;
                                        phaseone = 1;
                                        waiting_bots = 0;
                                        camera =0;
                                        }
                                }
                                else{}
                            }
                        }
                        else{}
                    }

            }

    }
    }
    cleanup_robot();
}

int main (int argc, char **argv) {
    #if SIMULATION
    #else
    ip = argv[1];
    #endif

    robot_setup();
    robot_loop();
}
```

Appendix B Challenge

B.1 Functions

The CHECK NODE function:

```
int check_node (int node_x, int node_y, int distance, int direction){
    struct maze_node *current_node = our_maze->nodes[node_x][node_y];
    if (current_node->distance < distance){
        return 0;
    } else {
        current_node->distance = distance;
        current_node->direction = (direction + 2) % 4;
        if (current_node->special == START){
            return 1;
        }

        // this calls the function recursively for every side where there is no wall
        for (int i = 0; i < 4; i++){
            if (current_node->walls[i] == 1){
                switch (i){
                    case NORTH:
                        check_node(node_x, node_y - 1, distance + 1, NORTH);
                        break;
                    case EAST:
                        check_node(node_x + 1, node_y, distance + 1, EAST);
                        break;
                    case SOUTH:
                        check_node(node_x, node_y + 1, distance + 1, SOUTH);
                        break;
                    case WEST:
                        check_node(node_x - 1, node_y, distance + 1, WEST);
                        break;
                    default:
                        break;
                }
            }
        }
        return 0;
    }
}
```

The CHECK WALL function:

```
void check_walls() {
    get_prox(IR_prox);
    switch(scan_state){
        case 0:
            offset = 0;
            scan_state = 1; //state to turn left
            our_maze->nodes[current_x][current_y]->walls[(current_dir + 2) % 4] = 1;
            break;
        case 1: //turn left first time
            set_speed(-NORM_SPEED, NORM_SPEED);
            offset++;
            if (offset > 16){
                set_speed(0, 0);
                offset = 0;
                scan_state = 2; //state to go forwards first time
            }
            break;
        case 2: //forward first time
            set_speed(NORM_SPEED, NORM_SPEED);
            offset++;
            if (offset > 20 || ((IR_prox[0] + IR_prox[7]) / 2 > PROX_STEP)){
                set_speed(0, 0);
                scan_wall((current_dir + 3) % 4);//scanns the wall with relative and absolute directions
                    accounted for
                scan_state = 3; //state to go backwards first time
            }
            break;
        case 3: //back first time
            set_speed(-NORM_SPEED, -NORM_SPEED);
            offset--;
            if (offset == 0){
                set_speed(0, 0);
                scan_state = 4; //state to turn right first time
            }
            break;
        case 4: //right first time
            set_speed(NORM_SPEED, -NORM_SPEED);
            offset++;
            if (offset > 16){
                set_speed(0, 0);
                offset = 0;
                scan_state = 5; //state to go forwards second time
            }
            break;
        case 5: //forward second time
            set_speed(NORM_SPEED, NORM_SPEED);
            offset++;
            if (offset > 45 || ((IR_prox[0] + IR_prox[7]) / 2 > PROX_STEP)){
                set_speed(0, 0);
                scan_wall(current_dir);//scans the wall with relative and absolute directions accounted for
                scan_state = 6; //state to go back second first time
            }
            break;
        case 6: //back second time
            set_speed(-NORM_SPEED, -NORM_SPEED);
            offset--;
            if (offset == 0){
                set_speed(0, 0);
                scan_state = 7; //state to turn right seccond time
            }
            break;
        case 7: //right seccond time
            set_speed(NORM_SPEED, -NORM_SPEED);
            offset++;
            if (offset > 16){
                set_speed(0, 0);
                offset = 0;
                scan_state = 8; //state to go forwards third time
            }
            break;
        case 8: //forward third time
            set_speed(NORM_SPEED, NORM_SPEED);
            offset++;
            if (offset > 40 || ((IR_prox[0] + IR_prox[7]) / 2 > PROX_STEP)){
                set_speed(0, 0);
                scan_wall((current_dir + 1) % 4);//scans the wall with relative and absolute directions accounted
                    for
                scan_state = 9; //state to go back third time
            }
            break;
        case 9: //back third time
            set_speed(-NORM_SPEED, -NORM_SPEED);
            offset--;
            if (offset == 0){
                set_speed(0, 0);
                scan_state = 10; //state to turn left seccond time
            }
            break;
        case 10: //turn left seccond time
            set_speed(-NORM_SPEED, NORM_SPEED);
            offset++;
```

```
            if (offset > 16){
                set_speed(0, 0);
                offset = 0;
                scan_state = 11; //ending state
            }
            break;
        case 11: //ending the function
            our_maze->nodes[current_x][current_y]->checked = 1;
            scan_state = 0;
            mode = TURN;
            break;
        default:
            break;
    }
}
```

The FIND DIR function:

```
void find_dir(){
    struct maze_node *current_node = our_maze->nodes[current_x][current_y];
    if (current_node->walls[(current_dir + 3 ) % 4] == 1){
        relative_dir = LEFT;
        current_dir = (current_dir + 3 ) % 4;
    } else if (current_node->walls[current_dir] == 1){
        relative_dir = FORWARD;
    } else if (current_node->walls[(current_dir + 1) % 4] == 1){
        relative_dir = RIGHT;
        current_dir = (current_dir + 1) % 4;
    } else {
        relative_dir = BACK;
        current_dir = (current_dir + 2) % 4;
    }
}
```

The FIND PATH function:

```
void find_path () {
    check_node(END_X, END_Y, 0, SOUTH);
    relative_path();
}
```

The FINISH PROGRAM function:

```
void finish_program() {
    find_path();
    if (offset < 16){
        set_speed(-NORM_SPEED, NORM_SPEED);
        offset++;
    } else if (offset < 40 && offset > 9){
        set_speed(NORM_SPEED, NORM_SPEED);
        offset++;
    } else {
        set_speed(0, 0);
        send_path();
        mode++;
    }
}
```

The FOLLOW LINE function:

```
void follow_line() {

    //standard mode to follow the line with a good positioning

    if (IR_ground[0] > LINE_STEP && IR_ground[1] > LINE_STEP && IR_ground[2] < LINE_STEP){
        set_speed(NORM_SPEED, NORM_SPEED);
    }else if (IR_ground[0] > LINE_STEP && IR_ground[1] > LINE_STEP && IR_ground[2] > LINE_STEP){
        set_speed(NORM_SPEED, 0);
    }else if (IR_ground[0] > LINE_STEP && IR_ground[1] < LINE_STEP && IR_ground[2] < LINE_STEP){
        set_speed(0, NORM_SPEED);
    }else if (IR_ground[0] < LINE_STEP && IR_ground[1] < LINE_STEP && IR_ground[2] < LINE_STEP){
        set_speed(NORM_SPEED, NORM_SPEED/1.5);
        if (ID == 1){
            switch (current_dir){
                case NORTH:
                    current_y = current_y - 1;
                    break;
                case EAST:
                    current_x = current_x + 1;
                    break;
                case SOUTH:
                    current_y = current_y + 1;
                    break;
                case WEST:
                    current_x = current_x - 1;
                    break;
                default:
                    break;
            }
            // once it comes back to the start it should stop
            if (our_maze->nodes[current_x][current_y]->special == START){
                set_speed(0, 0);
                mode = DONE;
                // once the node is checked it should go on to the next
            } else if (our_maze->nodes[current_x][current_y]->checked == 1){
                mode = TURN;
            } else {
                // if it isn't checked it should look for walls
                mode = SCAN;
            }
        }
            //go into turning mode;
        else {
            mode = TURN;
        }
    }
}
```

The NEW MAZE function:

```
void new_maze (){
    our_maze = (struct maze*) malloc(sizeof(struct maze));
    for (int x = 0; x < SZ_X; x++){
        for (int y = 0; y < SZ_Y; y++){
            our_maze->nodes[x][y] = new_node();
        }
    }
}
```

The RECEIVE PATH function:

```
void receive_path() {
    char message[MSG_LENGTH];
    receive_msg(message);
    if (to_int(message[0]) <= FINISH){
        path[corner_nr] = to_int(message[0]);
        corner_nr++;
    }
    if (to_int(message[0]) == FINISH){
        mode = FOLLOW;
    }
}
```

The RELATIVE PATH function:

```
int relative_path (){
    short int corner_nr = 0;
    short int current_x = START_X;
    short int current_y = START_Y;
    struct maze_node *current_node = our_maze->nodes[current_x][current_y];
    short int previous_direction = current_node->direction;
    // uses the direction to know which path to take
    switch (current_node->direction){
        case NORTH:
            current_y -= 1;
            break;
        case EAST:
            current_x += 1;
            break;
        case SOUTH:
            current_y += 1;
            break;
        case WEST:
            current_x -= 1;
            break;
        default:
            break;
    }
    current_node = our_maze->nodes[current_x][current_y];
    while (current_node->special != END){
        if (current_node->direction == previous_direction){
            path[corner_nr] = FORWARD;
        } else if ((previous_direction + 1) % 4 == current_node->direction){
            path[corner_nr] = RIGHT;
        } else if ((current_node->direction + 1) % 4 == previous_direction) {
            path[corner_nr] = LEFT;
        }
        corner_nr++;
        previous_direction = current_node->direction;
        switch (current_node->direction){
            case NORTH:
                current_y = current_y - 1;
                break;
            case EAST:
                current_x = current_x + 1;
                break;
            case SOUTH:
                current_y = current_y + 1;
                break;
            case WEST:
                current_x = current_x - 1;
                break;
            default:
                break;
        }
        current_node = our_maze->nodes[current_x][current_y];
    }
    path[corner_nr] = FINISH;
    return 0;
}
```

The ROBOT SETUP function :

```
void robot_setup() {
    init_robot();
    init_sensors();
    init_camera();
    init_communication();
}
```

The SEND PATH function:

```
void send_path() {
    char message[MSG_LENGTH];
    for (int i = 0; path[i] != FINISH; i++){
        message[0] = to_char(path[i]);
        send_msg(message);
    }
    message[0] = to_char(FINISH);
    send_msg(message);
}
```

The SCAN WALL function:

```
void scan_wall(short int direction){
    struct maze_node *current_node = our_maze->nodes[current_x][current_y];
    if ((IR_prox[0] + IR_prox[7]) / 2 > PROX_STEP)
    {
        current_node->walls[direction] = 0;
    } else {
        current_node->walls[direction] = 1;
    }
}
```

The TO CHAR function:

```
char to_char(int i){
    return (i + '0');
}
```

The TO INT function:

```
int to_int(char c){
    return (c - '0');
}
```

The TURN function:

```
void turn(){
    if(ID == 1){
        if (relative_dir == NONE){
            find_dir();
            printf("walls found for the current node %i, %i are: %i, %i, %i, %i. checked value: %i\n",
                   current_x,
                   current_y,
                   our_maze->nodes[current_x][current_y]->walls[0],
                   our_maze->nodes[current_x][current_y]->walls[1],
                   our_maze->nodes[current_x][current_y]->walls[2],
                   our_maze->nodes[current_x][current_y]->walls[3],
                   our_maze->nodes[current_x][current_y]->checked);
        }
    }
    else {
        relative_dir = path[corner];
    }
    if(IR_ground[0] > LINE_STEP && IR_ground[2] < LINE_STEP){
        if (ID == 1){
            mode = DRIVE;
            relative_dir = NONE;
        }
        else {
            mode = FOLLOW;
            corner++;
        }
        set_speed(0, 0);
    } else {
        switch (relative_dir){
            case LEFT:
                set_speed(-NORM_SPEED/3, NORM_SPEED);
                break;
            case FORWARD:
                set_speed(NORM_SPEED, NORM_SPEED);
                break;
            case RIGHT:
                set_speed(NORM_SPEED, NORM_SPEED/2.6);
                break;
            case BACK:
                set_speed(NORM_SPEED, -NORM_SPEED/4);
                break;
            case FINISH:
                mode = HALT;
                break;
            default:
                break;
        }
    }
}
```

B.2 Controller

```c
void robot_loop() {
    new_maze();

    current_x = START_X;
    current_y = START_Y;
    current_dir = NORTH;

    our_maze->nodes[START_X][START_Y]->special = START;
    our_maze->nodes[END_X][END_Y]->special = END;

    ID = get_robot_ID();

    if (ID = = 1){
        mode = SCAN;
     }  else {
        mode = RECEIVE ;
        path[0] = FORWARD;
     }

    while (robot_go_on()) {
        get_ground(IR_ground);
        switch (mode){
            case SCAN:
                 check_walls();
                break;
            case  RECEIVE:
                 receive_path();
                 break;
            case FOLLOW:
                 follow_line();
                 break;
            case DRIVE:
                follow_line();
                break;
            case TURN:
                turn();
                break;
            case DONE:
                finish_program();
                break;
            case HALT:
                set_speed(0, 0);
                break;
            default:
                printf("weird case, pls debug");
                break;
        }
    }
    cleanup_robot();
}

int main (int argc, char **argv) {
#if SIMULATION
#else
    ip = argv[1];
#endif


    robot_setup();
    robot_loop();
}
```

Appendix C FSM Diagrams

C.1 FSM - Task

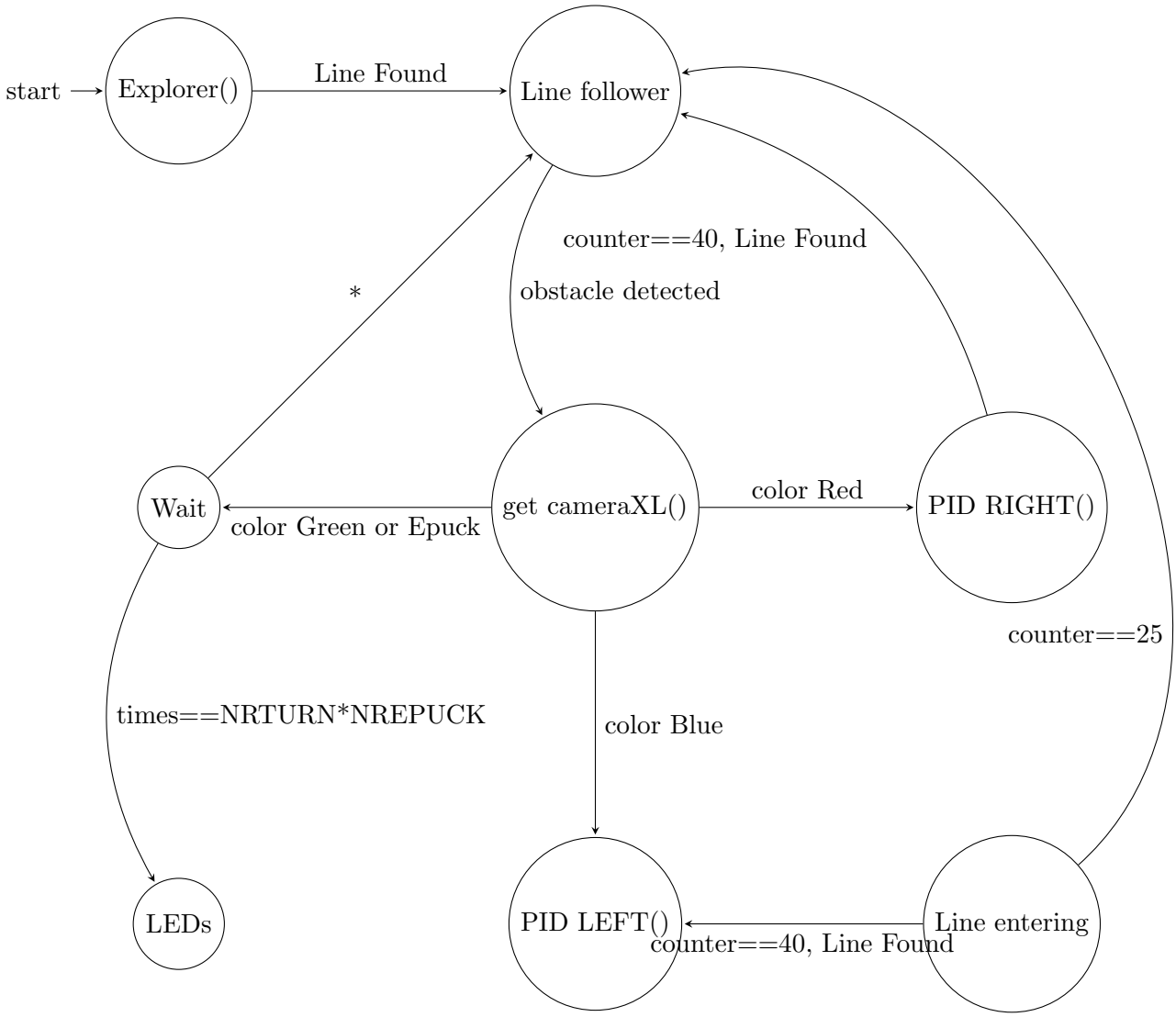

Figure C.1: FSM, Imposed Task

* If times=!NRTURN*NREPUCK and if Obstacle removed. If get camera XL was equal to EPUCK, another condition must be fullfilled : counter ==40.

C.2 FSM - Challenge

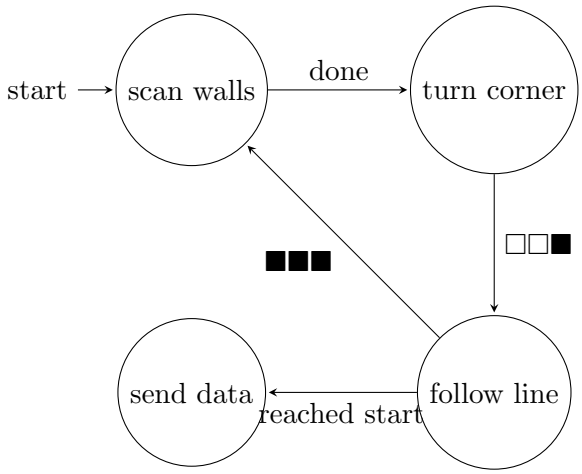

Figure C.2: state machine of the first robot